

Review: "Pylevin: Efficient Numerical Integration of Integrals Containing up to Three Bessel Functions"

Robert Reischke (2024) · Argelander-Institut für Astronomie, Universität Bonn

1. Introduction

Honestly, before reading this paper I had a pretty vague idea of what Bessel functions even were in a computational context. What Reischke is dealing with here is a problem that sounds niche but turns out to be surprisingly common in physics and cosmology: you need to numerically integrate functions that involve Bessel functions, and the standard tools just don't hold up. They oscillate too fast, quadrature methods struggle, and at some point the computation either crawls or fails entirely. The paper introduces `pylevin` – a Python package that tackles this using something called Levin's method – and the main pitch is that it handles not just one Bessel function (which other tools already do fine) but products of two or three, which is where things previously got messy.

2. Methodology

The math under the hood converts the integral into a differential equation problem, which neatly avoids the oscillation issue rather than trying to brute-force through it. The actual code is written in C++ for speed and wrapped in Python so it's usable without diving into low-level details. What I found interesting is the adaptive bisection – instead of splitting the integration interval uniformly, it focuses effort on the parts where the error is actually large. The benchmarking section compares `pylevin` against `hankel`, `hankl`, `pyffftlog` and `pyCCL`, which are all established tools in this space.

3. Results

For single Bessel function integrals, `pylevin` keeps up with the specialized packages, usually within a factor of two – not faster, but competitive. The real story is with two or three Bessel functions: `pylevin` is up to four orders of magnitude faster than standard adaptive quadrature, and more importantly, it stays stable in regimes where quadrature just gives up entirely. The author is upfront about where competitors win, which actually makes the results feel more trustworthy rather than oversold.

4. Key Insights

Insight 1 – The homogeneous solution reuse genuinely surprised me. In many real workflows, like running an MCMC chain, the structure of the integral stays fixed while only $f(x)$ changes. `Pylevin` can precompute the expensive part once and reuse it, which apparently speeds things up by an order of magnitude. That's the kind of optimization that only makes sense if you've actually used these methods in practice, not just benchmarked them in isolation.

Insight 2 – There's real value in being general even if you're not always the fastest. Every other tool the paper compares against is faster in its specific niche. But they're all locked into particular integral forms. `Pylevin` trades a bit of speed in the easy cases for the ability to handle a much wider range of problems without restructuring

the math. For research use, that flexibility matters a lot.

Insight 3 – Seeing `scipy.integrate.quad` fail completely – not just run slowly, but genuinely not converge – was a good reminder that default tools have limits that aren't always obvious. It's easy to trust standard library functions without thinking about where they break down. This paper made that concrete.

5. Conclusion

In my opinion, Pylevin is not trying to replace every specialized tool out there, and it doesn't pretend to. What it does is fill a gap that was genuinely empty: a general, accessible method for integrals with two or three Bessel functions that actually runs in reasonable time. The paper is honest about trade-offs and the benchmarks feel fair. Where it could go next is probably toward higher-dimensional cases or smarter ways to guide the bisection using properties of the integrand – but even as it stands, it seems like something worth knowing about if you work anywhere near this kind of numerical problem.